Data Structure Based Puzzle (By Dr. Ranjit Rajak)

Write Algorithm and explain with numerical example of the following puzzles.

1. Design a stack data structure that supports the following four operations.
   - **push(x):** Insert an element x onto the top of the stack.
   - **pop():** Remove and return the element at the top of the stack.
   - **findMiddle():** Retrieve the middle element of the stack without removing it.
   - **deleteMiddle():** Remove the middle element from the stack.
2. Given an array **height[]** which represents the height of **n** people standing in a line. A person **i** can see a person **j** if **height[j] < height[i]** and there is no person **k** standing in between them such that **height[k] >= height[i]**. Find the maximum number of people a person can see.
3. Given an array **arr[]** of size **n**, the task is to count the number of unique differences between the two maximum elements of every subarray of size at least 2 of the given array.
4. Given a source stack, copy the contents of the source stack to the destination stack maintaining the same order without using extra space.

   Examples:
   Input: *st = [3, 2, 1]*
   Output: *Clone : [3, 2, 1]*
   Input: *st = [10, 5, 4]*
   Output: *Clone = [10, 5, 4]*

5. Given **q** queries, You task is to implement the following **four** functions for a stack:

   - **push(x) –** Insert an integer x onto the stack.
   - **pop() –** Remove the top element from the stack.
   - **peek() -** Return the top element from the stack. If the stack is empty, return -1.
   - **getMin() –** Retrieve the minimum element from the stack in O(1) time. If the stack is empty, return -1.

   Each query can be one of the following:

   1 x : Push x onto the stack.
   2 : Pop the top element from the stack.
   3: Return the top element from the stack. If the stack is empty, return -1.
   4: Return the minimum element from the stack.